



REPORTE FINAL DEL PROYECTO INTEGRADOR

Sistema de Cafetería desarrollado en Python

Curso: Solución de problemas con programación computacional

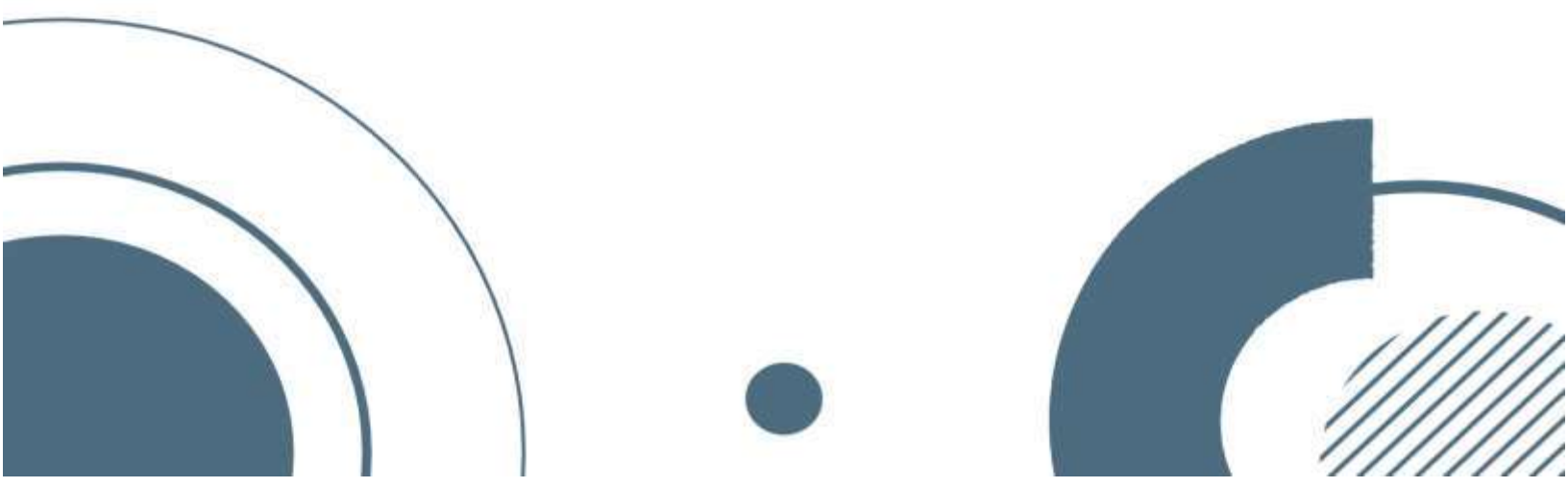
Actividad: Entrega Final del Proyecto / Presentación Final

Estudiante: Evelyn Itzel León Uribe

Matrícula: AL07311179

Docente: Ricardo Alfredo Monroy Rodríguez

Fecha: Viernes 25 de Septiembre de 2026



1. Introducción

El presente reporte documenta el proyecto integrador de un sistema de cafetería desarrollado en Python. El programa busca resolver una problemática básica de operación: registrar ventas, controlar existencias, realizar un corte de caja y conservar información mediante archivos de texto.

De acuerdo con la actividad, el proyecto final debe integrar manejo de errores, persistencia en archivos externos, control de inactividad y depuración técnica.

2. Problemática que resuelve

Una cafetería necesita llevar un control sencillo de sus productos, ventas e inventario. Si estos datos solamente se mantienen durante la ejecución del programa, pueden perderse al cerrarlo. Por ello, el sistema utiliza archivos .txt para conservar información de ventas, productos, usuarios y reportes.

Objetivo general

Desarrollar un sistema de consola que permita administrar operaciones básicas de una cafetería mediante Python, utilizando estructuras de control, funciones, colecciones, archivos de texto, excepciones y herramientas de depuración.

3. Reglas principales del sistema

- El programa solicita el nombre o nickname del usuario al iniciar.
- Se captura la fecha de operación y se almacena en una tupla.
- Solo se puede registrar una venta cuando existe stock suficiente.
- La cantidad comprada debe ser mayor que cero y no superar las existencias.
- Una venta válida disminuye el stock y registra la operación en ventas.txt.
- El corte de caja muestra la cantidad de ventas y el total acumulado.
- Después del corte, el inventario se reabastece a las cantidades iniciales.
- Los archivos pueden leerse y también pueden recibir notas o registros adicionales.

4. Diseño algorítmico: Entrada, Proceso y Salida

Entrada	Proceso	Salida
Usuario, fecha, opción del menú, producto, cantidad y texto para archivos.	Validar datos, calcular total, actualizar stock, leer/escribir archivos, manejar excepciones y depurar.	Mensajes en consola, stock actualizado, ventas registradas, reportes y contenido de archivos.

5. Cumplimiento de los requerimientos técnicos

1. Identificación de usuario

La función `pedir_usuario()` solicita el nombre o nickname mediante `input()` y guarda el resultado en la variable `usuario`.

2. Bienvenida dinámica

El sistema construye un mensaje usando el nombre capturado: «¡Hola, [usuario]! Bienvenido/a al sistema.»

3. Pantalla de carga

La función `pantalla_carga()` muestra cinco pasos de carga y utiliza `time.sleep(1)` en cada paso, para una duración máxima de 5 segundos.

4. Menú como matriz y while

La variable `matriz_menu` organiza las seis opciones en dos columnas y tres filas. El ciclo `while` mantiene el menú activo hasta seleccionar la opción 6.

5. Control de inactividad

La función `medir_inactividad()` utiliza un ciclo `for` de 1 a 10 y posteriormente pregunta si el usuario desea continuar con «si» o regresar por inactividad con cualquier otra respuesta. La implementación actual simula los minutos usando `time.sleep(0.1)`.

6. Fecha en tupla

La función `pedir_fecha()` captura día, mes y año y los almacena como `Fecha = (dia, mes, anio)`. Esta información se utiliza al escribir en los archivos.

7. Persistencia en archivos

El sistema utiliza `ventas.txt`, `productos.txt`, `usuarios.txt` y `reportes.txt`. Se emplean los modos `r`, `w` y `a` según la operación requerida.

8. Control de excepciones

Se utilizan bloques `try-except` para `FileNotFoundError`, `PermissionError` y `ValueError` en las operaciones correspondientes.

9. Depuración con PDB

El programa importa el módulo pdb y contiene preparada la instrucción `pdb.set_trace()`. Para documentar el debugging se debe ejecutar con esta instrucción activa y registrar las variables y el flujo revisado.



10. Comentarios de calidad

El código incluye comentarios de orientación, aunque para cumplir al máximo el criterio de comentarios extensos conviene ampliar la documentación de cada función y de las partes importantes del flujo.

6. Persistencia y manejo de archivos

El sistema trabaja con cuatro archivos de texto:

Archivo	Uso	Operaciones
ventas.txt	Guardar las ventas realizadas.	Anexar (a), lectura (r)
productos.txt	Conservar el inventario actual.	Escritura/actualización (w), lectura (r)
usuarios.txt	Registrar el inicio de sesión.	Anexar (a), lectura (r)
reportes.txt	Guardar cortes de caja y notas.	Anexar (a), lectura (r)

La sentencia `with open(...)` se utiliza para trabajar con los archivos y permitir que sean cerrados automáticamente al terminar la operación.

7. Manejo de excepciones

El programa incorpora `try-except` para controlar errores comunes y mostrar mensajes comprensibles al usuario.

- `FileNotFoundError`: se utiliza cuando un archivo solicitado no existe.
- `PermissionError`: controla problemas de permisos de lectura o escritura.
- `ValueError`: controla entradas que no pueden convertirse al tipo numérico esperado.
- Las excepciones evitan que ciertos errores de ejecución terminen abruptamente el programa.

8. Depuración técnica con PDB

PDB es el depurador estándar de Python. Permite detener la ejecución del programa en un punto determinado y revisar el estado de las variables y el flujo de ejecución.

En el proyecto se prepara el uso de PDB mediante:

```
import pdb pdb.set_trace()
```

Para realizar la evidencia de depuración se recomienda:

1. Descomentar la línea `pdb.set_trace()` dentro del flujo principal.
2. Ejecutar el programa y llegar al punto donde está el breakpoint.
3. Utilizar `p` para consultar variables, por ejemplo: `p opcion` o `p usuario`.
4. Utilizar `n` para avanzar una línea y observar el flujo.
5. Utilizar `s` para entrar en una función cuando sea necesario.
6. Utilizar `c` para continuar la ejecución.
7. Registrar mediante capturas la consola y explicar qué variable o comportamiento se revisó.
8. Documentar la falla encontrada y la corrección realizada.

Importante: en la versión proporcionada, `pdb.set_trace()` está comentado. Por lo tanto, la evidencia de debugging debe generarse ejecutando realmente PDB antes de entregar, si se desea demostrar el requisito.

9- Conclusiones

El proyecto integra diferentes elementos estudiados durante el curso en una aplicación de consola. El uso de funciones permite dividir las tareas del sistema, mientras que las listas, diccionarios y tuplas permiten organizar los datos. Los archivos de texto proporcionan persistencia y los bloques `try-except` permiten controlar varios errores de ejecución.

La depuración con PDB complementa el desarrollo porque permite observar el estado interno del programa durante su ejecución. Como mejoras futuras se podría implementar una base de datos, autenticación de usuarios, validación más completa de fechas y un mecanismo de inactividad basado en tiempo real..

10. Referencia de los requerimientos de la actividad

Este reporte se elaboró tomando como base las instrucciones proporcionadas para la Actividad Evaluable 7 — Entrega Final del Proyecto (Fase II) y la Actividad Evaluable 8 — Presentación Final o Examen. La Actividad 7 establece los diez requerimientos técnicos y los

entregables del proyecto; la Actividad 8 establece la estructura de la exposición, la demostración en vivo y la explicación de la solución técnica.